

LENGUAJE PHP (PARTE II)

Hasta ahora hemos visto...

- Variables (diferentes tipos: de referencia, de valor, globales, ...), constantes y comentarios
- Convertir un tipo de dato a otro (int a double, ...)
- Operaciones
- Condicionales (if y switch)
- Bucles (for, while y do while)
- Location

Ahora vamos a ver...

- Formularios
- Inclusión de ficheros
- Arrays (distintos tipos y formas de recorrerlos) y String como arrays
- Funciones
- Para saber más: Funciones de cifrado

Uso de formularios HTML desde PHP

ENVÍO DE DATOS DESDE UN FORMULARIO

Todos los controles del formulario deben estar dentro de un elemento **form**, que nos permite configurar:

- **method**: puede ser **get** o **post** (por defecto get) y se refiere a la forma en la que se pasan los datos
- **action**: indica la URL a la dirección del servicio que recoge los datos

Dentro del elemento form, se colocan los elementos HTML capaces de recoger información: **input**, **option**, **textarea**

Estos elementos poseen, entre otros, los siguientes atributos:

- **name**: indica el nombre de la variable que se envía al servidor
- **value**: inicialmente, permite indicar el valor que se asigna a dicha variable, luego puede ser modificado por el usuario del formulario

ENVÍO DE DATOS DEL FORMULARIO MEDIANTE GET (1/3)

```
<form action="recogida1.php" method="get">
  <label for="nombre">Escriba su nombre</label>
  <input type="text" name="nombre" /> <br />
  <label for="apellidos">Escriba sus apellidos</label>
  <input type="text" name="apellidos" /> <br />
  <label for="nombre">Escriba su dirección</label>
  <input type="text" name="direccion" /> <br />
  <label for="nombre">Escriba sus teléfonos:</label>
  <br />
  <input type="text" name="telefono1" maxlength="9"
    pattern="[0-9]{9}" /> <br />
  <input type="text" name="telefono2" maxlength="9"
    pattern="[0-9]{9}" /> <br />
  <input type="submit" value="enviar" /> <br />
</form>
```

En pantalla



Escriba su nombre

Escriba sus apellidos

Escriba su dirección

Escriba sus teléfonos:

ENVÍO DE DATOS DEL FORMULARIO MEDIANTE GET (2/3)

¿Qué se genera al pulsar Enviar en el código anterior?

La forma de una URL tipo GET es: <http://urlpagina?nombre1=valor1&nombre2=valor2&nombre3=valor3&...>

Se genera la URL (en nuestro caso caso):

<http://localhost/recogida1.php?nombre=Lorena&apellidos=Castellanos+Cacho&direccion=Calle+Alegr%C3%ADa&telefono1=612345678&telefono2=698705432>

Alegría se convierte en *Alegr%C3%Ada* porque en la URL no puede haber caracteres fuera del código ASCII y el carácter se recodifica usando los dígitos Unicode dentro del operador %.

La página *recogida1.php* es la siguiente:

```
<!DOCTYPE HTML>
<html lang="es-ES">
<head>
  <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
  <title></title>
</head>
<body>
  <pre>
    <?php
      print_r ($_GET);
    ?>
  </pre>
</body>
</html>
```

Lo que
recogería con
nuestros
datos



Array

```
(
  [nombre] => Lorena
  [apellidos] => Castellanos Cacho
  [direccion] => Calle Alegría
  [telefono1] => 612345678
  [telefono2] => 698705432
)
```

ENVÍO DE DATOS DEL FORMULARIO MEDIANTE GET (3/3)

IMPORTANTE: Para recoger un valor concreto, por ejemplo, el valor de “nombre”, utilizaríamos `$_GET[“nombre”]`
¿Qué ocurriría si no lo hubiéramos recibido? ¡Error undefined index! Por esto, **SIEMPRE COMPROBAR SI EXISTE** con `isset -> if(isset($_GET[“nombre”]))`

El paso de datos de un formulario mediante GET tiene estas connotaciones:

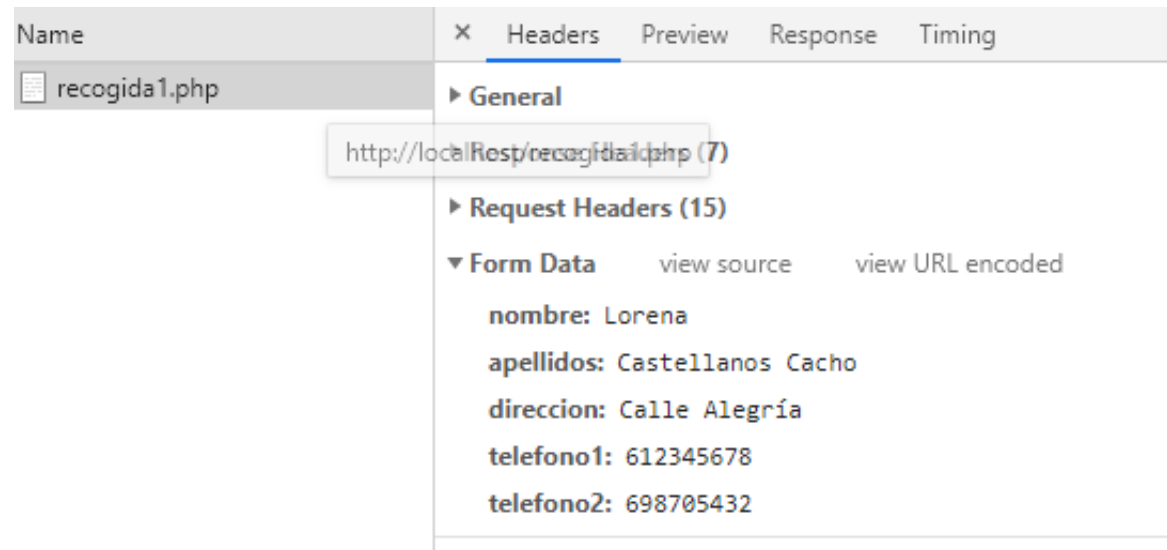
- Los pares de datos nombre/valor están a la vista, ¡no pasar contraseñas y otros datos críticos!
- Las páginas generadas con GET pueden añadirse como marcador o favorito a los navegadores y páginas de marcadores. Eso puede ser interesante, pero sólo debe ser posible si la URL no contiene datos que hagan que otro usuario que obtenga esa URL pueda acceder a información confidencial
- No tenemos que pasar por el formulario para enviar valores a la página receptora, ya que en la propia barra de navegación podemos escribir los valores y nombres a mano.
- La depuración de nuestros programas es más cómoda con GET (vemos lo que llega del formulario en la propia URL). Por ello, se suele trabajar pasando parámetros con GET durante el desarrollo de la aplicación y hacerlos con POST en la versión final

ENVÍO DE DATOS DEL FORMULARIO MEDIANTE POST

Los formularios se crean igual en ambos casos, pero ahora cambiamos “method” (de form) con el valor “post”:

```
<form action="recogida1.php" method="post">
```

Mediante POST, los nombres y valores del formulario no viajan en la URL, sino que se colocan en la cabecera del paquete http; por lo que les dotamos de una menor visibilidad. Podemos verlos con herramientas que nos permitan examinar las cabeceras http del paquete que se envía por la red. Por ejemplo, utilizando el apartado **Network** de las **DevTools** de Chrome:



RECEPCIÓN DE DATOS DE UN FORMULARIO DESDE UNA PÁGINA PHP

PHP permite recibir los datos pasados por los parámetros usando dos variables, que son en realidad dos arrays que contendrán todos los valores del formulario. Los arrays se llaman `$_GET` y `$_POST`, cada uno de ellos dedicado a almacenar los datos enviados con el método correspondiente.

`$_REQUEST`: recoge los valores sin importar si se han pasado por GET o por POST

Los controles del formulario envían los valores asociados al nombre que dimos a ese control, explicitado en el atributo **name**. Por ejemplo, si el método es POST, se accederá al valor mediante `$_POST["name"]`

¡¡COMPROBAR SIEMPRE SI EXISTE LA VARIABLE Y, EN CASO DE QUE EXISTA, SI ESTÁ VACÍA (EMPTY) !!

RECEPCIÓN DE DATOS DE UN FORMULARIO DESDE UNA PÁGINA PHP

A veces se usa la misma página en la que está el formulario para recibir datos del mismo (la página sería ahora php)

```
<!DOCTYPE html>
<html lang="es">
  <head>
    <meta charset="UTF-8" />
    <title>Ejemplo de formulario en la propia página</title>
  </head>
  <body>
    <?php
    if(isset($_POST["nombre"]) && isset($_POST["contra"])) {
//si no se reciben nombre y contraseña, se muestra el formulario
?>

    <!--Este código HTML está dentro del IF de PHP -->
    <form action="form1.php" method="post">
      <label for="nombre">Escriba su usuario</label>
      <input type="text" name="nombre" maxlength="20" /><br />
      <label for="contra">Escriba la contraseña</label>
      <input type="password" name="contra" maxlength="20" />
      <br />
      <input type="submit" value="enviar"/><br />
    </form>
```

```
<?php //cierre del if
    else { //Se han pasado datos, se procesan
      $nombre=$_POST["nombre"];
      $contra=$_POST["contra"];
      if($nombre!="Jorge" || $contra!="123456") {
        echo "No es válido ese usuario y contraseña<br>";
        echo "<a href='form1.php'>Volver al formulario</a>";
      }
      else echo "Entrada correcta";
    }
  } ?>
</body> </html>
```

Inclusión de ficheros

```
include("funciones.php");
/*copia y pega el código del archivo
funciones.php tal cual.
Si no lo encuentra, SEGUIRÁ ejecutando
el archivo aunque muestre advertencia
*/
```

```
require("funciones.php");
/*copia y pega el código del archivo
funciones.php tal cual.
Si no lo encuentra, generará un
error grave y PARARÁ la
ejecución del código
*/
```

Arrays

ARRAYS ESCALARES (1/2)

Son un conjunto de valores a los que se accede a través de un **índice**, que es un número entero en el que el primer elemento tiene índice cero. Así, por ejemplo, unas notas se podrían almacenar de esta forma:

Acceder a una nota cualquiera requiere conocer su índice.

No hace falta declarar el array, simplemente se utiliza.

Los datos pueden ser de distinto tipo:

```
$valor[0] =18;
$valor[1]="Hola";
$valor[2]=true;
$valor[3]=3.4;
```

```
$nota[0]=7;
$nota[1]=8;
$nota[2]=6;
$nota[3]=6;
$nota[4]=5;
$nota[5]=4;
```


ARRAYS ESCALARES (2/2)

Si no ponemos índice, éstos se van asignando automáticamente empezando por el cero

```
$valor[] =18;
$valor[]="Hola";
$valor []=true;
$valor[]=3.4;
```

Podemos asignar valores a un array con la función **array()**

```
$numero=array(17, 34, 45, 2, 9, -5, 7);
```

PARA RECORRER ARRAYS BUCLE FOR:

```
$valor[0]=mt_rand(1,10);
$valor[1]=mt_rand(1,10);
$valor[2]=mt_rand(1,10);
...//iiiiimucho trabajo!!!!
```



```
for($i=0;$i<1000;$i++)
    $valor[$i]=rand(1,10);
```

TAMAÑO DE UN ARRAY: función **count(\$array)**

```
for($i=0;$i<count($valor);$i++){
    echo $valor[$i]."<br />";
}
```


ARRAYS ASOCIATIVOS

El índice es un texto que se comporta como una clave que permite acceder al dato. Los arrays así contruidos forman estructuras de datos de tipo clave/valor, como ocurre con los arrays **\$_GET** Y **\$_POST**

```
$nota["Antonio"]=5;
$nota["Luis"]=9;
$nota["Ana"]=8;
$nota["Eloy"]=5;
$nota["Gabriela"]=6;
$nota["Berta"]=7;
```

Son más legibles pero NO permiten su manejo mediante los bucles clásicos

Se relacionan muy bien con las bases de datos

Lo mismo que...

```
$nota=array("Antonio"=>5,"Luis"=>9,"Ana"=>8,"Eloy"=>5,
"Gabriela"=>6,"Berta"=>7);
```

BUCLE FOREACH (1/2)

Recorrer arrays asociativos o incluso escalares con elementos sin inicializar resulta muy complejo con **while** o **for**. El bucle **foreach** permite recorrer todos los elementos de un array sin importar si hay elementos indefinidos, ya que es capaz de ignorarles (idóneo para recorrer arrays asociativos)

```
foreach(array as índice=>valor ){
    sentencias
}
```

-  **array** es el nombre del array que se va a recorrer
-  **índice** es la variable que recogerá el índice de cada elemento a medida que se recorra el array
-  **valor** es la variable que irá recogiendo el valor de cada elemento del array

BUCLE FOREACH (2/2)

```
$nota=array("Antonio"=>5, 4, 7, 5=>6, "Luis"=>9, 2,"Berta"=>7);
foreach ($nota as $i=>$v){
    echo "La nota de $i es $v\n";
}
/* Escribe:
    La nota de Antonio es 5
    La nota de 0 es 4
    La nota de 5 es 6
    La nota de Luis es 9
    La nota de 6 es 2
    La nota de Berta es 7
*/
```

ARRAYS MULTIDIMENSIONALES (1/3)

Sirve para representar datos agrupados en una estructura que utiliza dos índices para representar cada elemento del array. El array puede ser asociativo y se pueden utilizar más de dos dimensiones

```
$nota[0][0]=9;
$nota[0][1]=7;
$nota[1][0]=9;
...
```

```
$poblacion["España"]["Castilla y León"]["Palencia"]=80000;
$poblacion["España"]["Castilla y León"]["Valladolid"]=370000;
$poblacion["España"]["Asturias"]["Oviedo"]=115000;
$poblacion["Alemania"]["Brandemburgo"]["Berlin"]=4000000;
```

```
$poblacion["Alemania"]["Berlin"]=4000000;
$poblacion["Alemania"]["Hanover"]=1200000;
$poblacion["Francia"]["Paris"]=7400000;
$poblacion["Francia"]["Lyon"]=2300000;
```

ARRAYS MULTIDIMENSIONALES (2/3)

De esta forma podemos considerar a los arrays como una especie de almacenes de datos que se parecen mucho a las tablas de las bases de datos relacionales. Otra forma posible es:

```
$alemania=array("Berlin", "Hannover");
$francia=array("París", "Lyon");
$espania=array("Palencia");
$poblacion=array("alemania"=>$alemania,
                 "francia"=>$francia,"espania"=>$espania);
```

CONSTRUCTOR ARRAY MULTIDIMENSIONAL

```
$poblacion = array(
    "España" => array(
        "Palencia"=>80000,
        "Valladolid" => 35000,
        "Oviedo" => 120000
    ),
    "Francia" => array(
        "París"=>7000000,
        "Lyon"=>2100000
    )
);
```

Lo mismo que...

```
$poblacion["España"]["Palencia"]=80000;
$poblacion["España"]["Valladolid"]=350000;
$poblacion["España"]["Oviedo"]=120000;
$poblacion["Francia"]["Paris"]=7000000;
$poblacion["Francia"]["Lyon"]=2100000;
```

ARRAYS MULTIDIMENSIONALES (3/3)

RECORRIDO MEDIANTE FOREACH DE ARRAYS MULTIDIMENSIONALES

Se basa en la idea de considerar que un array multidimensional es un array de arrays

```
$poblacion=array(
    "España"=>array(
        "Palencia"=>80000,
        "Valladolid"=>350000,
        "Oviedo"=>120000
    ),
    "Francia"=>array(
        "Paris"=>7000000,
        "Lyon"=>2100000
    )
);
foreach($poblacion as $i=$valor){
    echo "El índice $i vale $valor<br />";
}
```

El resultado de este código sería:

El índice España vale Array

El índice Francia vale Array

```
$poblacion=array(
    "España"=>array(
        "Palencia"=>80000,
        "Valladolid"=>350000,
        "Oviedo"=>120000
    ),
    "Francia"=>array(
        "Paris"=>7000000,
        "Lyon"=>2100000
    )
);
foreach ($poblacion as $pais => $ciudades) {
    foreach ($ciudades as $ciudad => $valor) {
        echo "<p>País: $pais, Ciudad: $ciudad,".
            " población = $valor</p>";
    }
}
```

Este código nos permite examinar cada valor

Obtiene de resultado:

```
País: España, Ciudad: Palencia, población = 80000
País: España, Ciudad: Valladolid, población = 350000
País: España, Ciudad: Oviedo, población = 120000
País: Francia, Ciudad: Paris, población = 7000000
País: Francia, Ciudad: Lyon, población = 2100000
```


INSPECCIÓN DE ARRAYS MEDIANTE FUNCIONES DE RECORRIDO

FUNCIÓN	USO
current(array)	Devuelve el valor del elemento al que actualmente señala el puntero. Si no hay ningún elemento, devuelve false
key(array)	Devuelve la clave a la que señala el puntero
next(array)	Mueve el puntero al siguiente elemento del array y devuelve su valor. Si el elemento actual es el último, next devuelve false

FUNCIÓN	USO
prev(array)	Mueve el puntero al elemento anterior del array y devuelve su valor. Si el elemento actual es el primero, prev devuelve false
reset(array)	Coloca el puntero en el primer elemento del array y devuelve su valor. Si no hay ningún elemento en el array, devuelve false
end(array)	Coloca el puntero en el último elemento y devuelve su valor

USO DE ARRAYS EN FORMULARIOS (Ver ejemplo completo en el archivo *ejemploArrays.php* del moodle)

Cuando tenemos varios campos de formulario que queremos unir para que se envíen como un array, simplemente **necesitamos colocarle unos corchetes al final del nombre** (atributo "name") del campo.

```
<form action="#" method="POST">
    <input type="text" name="campo[]" value="campo1">
    <input type="text" name="campo[]" value="campo2">
    <input type="text" name="campo[]" value="campo3">
    <input type="submit" value="enviar">
</form>
```

Tiene 3 campos input de "type text" que se llaman igual, lo que en principio podría parecer un error. Sin embargo, todos ellos tienen unos corchetes al final del "name", por lo **que el navegador los enviará como si fuera un array**, en vez de enviarlos como campos "suelos". Para **recibir los datos**, tenemos que tratarlos como un array, por lo que sería algo así como lo siguiente:

```
foreach($_POST['campo'] as $campo) {
    echo "<p>Valor recibido: $campo</p>";
}
```

Strings

Una serie de caracteres o texto. Para asignar valores a un string basta con utilizar el operador “=”. En el caso de los caracteres especiales sólo se pueden indicar dentro de comillas dobles

```
$string1="Soy un texto";
$string2="Yo también";
$string2.=" y sigo siendo un texto"; //se concatena al $string2 y ahora éste vale "Yo también y sigo siendo un texto"
echo "yo {$string1}chulo" //muestra "yo Soy un textochulo"
```

No se admite el uso de foreach para recorrer Strings ni tampoco funcionarían ninguna de las funciones de arrays. Si hacemos \$string1[3]="s"; escribirá en la tercera posición una "s", es decir, "Sos un texto"

```
$nombre="Jorge";
$texto=<<

```

CADENAS NOWDOC →

→ CADENAS HEREDOC

```
$nombre="Jorge";
$texto=<<<'fin'
Mi querida amiga <br />
escribo estas líneas esperando que me leas. <br />
Firmado: $nombre <br />
fin;
echo $texto;
```


FUNCIONES PHP

1. Poseen un **nombre** o **identificador** que cumple las reglas indicadas para los demás identificadores que conocemos (como los de las variables). Pero, a diferencia de las variables, **NO UTILIZAN \$**. Se aconseja que su nombre se escriba en minúsculas. El identificador de una función siempre va seguido de paréntesis
2. Al definir las funciones, **entre paréntesis se indican los parámetros** que necesita la función para hacer su labor (si no necesita ninguno, no ponemos nada entre paréntesis)
3. Tras la declaración de los parámetros se escribe el **código de la función**, en el que se implementa lo que la función debe ser capaz de realizar
4. Las funciones pueden devolver un valor, resultado del trabajo de la misma, mediante el comando **return**

Declaración de una función

Seguimos la sintaxis

```
function nombreDeLaFunción(listaDeParámetros){  
    código de la función  
}
```

EJEMPLO DE FUNCIÓN SIN PARÁMETROS

```
function BuenosDias (){
    $var = "Buenos días bonito";
    echo $var."\n";
}
//Esta función muestra por pantalla
//"buenos días bonito"
```

EJEMPLO DE FUNCIÓN CON PARÁMETROS

```
function BuenosDiasBis ($numRepeticiones) {
    for($i=0;$i<$numRepeticiones;$i++){
        BuenosDias();
        //llama a la función BuenosDias() que
        //muestra por pantalla "Buenos días bonito"
    }
}
//Esta función muestra "Buenos días bonito" tantas veces
//como sea el número que almacene $numRepeticiones
//Es decir, si $numRepeticiones es 5, lo mostrará 5 veces
```

EJEMPLOS DE LLAMADAS A ESTAS FUNCIONES:

```
BuenosDias();
BuenosDiasBis(5); //Escribirá 5 veces "Buenos días bonito"
```

Las funciones se pueden declarar en cualquier parte de una página PHP, pero lo aconsejable es declarar las funciones en la cabecera de la página web o incluso por encima del inicio del código HTML para facilitar su mantenimiento

Paso de parámetros por referencia

Por defecto, las funciones reciben los parámetros **POR VALOR**: los parámetros de la función usan copias de los valores

```
function f1($x){
    $x=9;
}
$x=7;
f1($x);
echo $x; //escribe 7
```

Este es un ejemplo de código confuso. La función f1 se ha definido de modo que requiere un parámetro al que ha llamado \$x y que dentro de la función toma el valor 9. Por otro lado, fuera de la función se ha definido una variable que también se llama \$x que vale 7. Cuando se invoca a f1 con f1(\$x), lo que hace ésta es tomar una copia de ese valor. El cambio que se realiza dentro de la función no afecta a la variable exterior a ella. Por ello, cuando mostramos \$x con echo \$x lo que muestra es un 7 (y no un 9)

Si queremos que una función modifique los valores originales tenemos que pasarle los parámetros **POR REFERENCIA**: lo indicamos con el signo & antes del nombre del parámetro

```
function swap(&$x,&$y){
    $aux=$x;
    $x=$y;
    $y=$aux;
}
$valor1=12;
$valor2=17;
swap($valor1,$valor2);
echo $valor1." ".$valor2;
```

Escribe 17 12 ya que ahora sí que las variables originales han sido modificadas. **CUANDO UNA FUNCIÓN UTILIZA PASO POR REFERENCIA AL INVOCARLA DEBE RECIBIR NOMBRES DE VARIABLES Y NO EXPRESIONES O VALORES LITERALES.** Lo siguiente daría error porque la función swap espera variables y NO expresiones:

```
swap(9,8); //error: Se requiere una variable
```

Parámetros predefinidos

Se puede indicar un valor por defecto a los parámetros a fin de permitir que dicho parámetro sea opcional; de modo que, si se pasa el parámetro se toma el valor que pasa y si no, se toma el opcional por defecto

```
function potencia($base,$exponente=2){
    $valor=1;
    for($i=1;$i<=$exponente;$i++){
        $valor*=$base;
    }
    return $valor;
}
echo potencia(2,3); //Escribe 8, es decir 2³
echo potencia(4);   //Escribe 16, es decir 4²
```

La segunda vez que se invoca a la función **\$exponente** toma el valor por defecto 2

Variables globales (Ver [Ejemplo de funciones con variables](#) en el aula virtual)

Las variables globales permiten ser utilizadas en todo el código, sin que las funciones utilicen variables que se han definido fuera de la función

```
function encadenar($numero,$carácterRelleno){

    global $texto;
    for($i=1;$i<=$numero;$i++){
        $texto.=$carácterRelleno;
    }
}
$texto="Hola";
```

```
encadenar(12,"d");
echo $texto."<br />"; //Escribe: Holadddddddddddddd
encadenar(9,"+-");
echo $texto."<br />"; //Escribe: Holadddddddddddddd+-----
```

NO RECOMENDABLE ¡¡Las funciones deben ser independientes!!

Variables estáticas

Son variables locales a la función (sólo se podrán utilizar dentro de la función), pero que son capaces de recordar su valor entre cada llamada a la función

```
function estatica(){  
    static $cuenta=0;  
    $cuenta++;  
    echo "Esta es la llamada número $cuenta<br />";  
}  
  
for($i=1;$i<=10;$i++){  
    estatica();  
}
```

Salida del
código



```
Esta es la llamada número 1  
Esta es la llamada número 2  
Esta es la llamada número 3  
Esta es la llamada número 4  
Esta es la llamada número 5  
Esta es la llamada número 6  
Esta es la llamada número 7  
Esta es la llamada número 8  
Esta es la llamada número 9  
Esta es la llamada número 10
```

El código de la función escribe el valor de la variable **\$cuenta**. Esta variable debería eliminarse al abandonar la función, pero sobrevive y se puede seguir utilizando. Eso sólo es posible gracias a la línea en la que se define la variable como estática (**static \$cuenta=0**) que sólo se ejecuta en la primera llamada. Si en esa línea eliminamos la palabra static, entonces la variable se crea en cada llamada y las diez líneas anteriores, escribirían lo mismo (*Esta es la llamada número 1*)

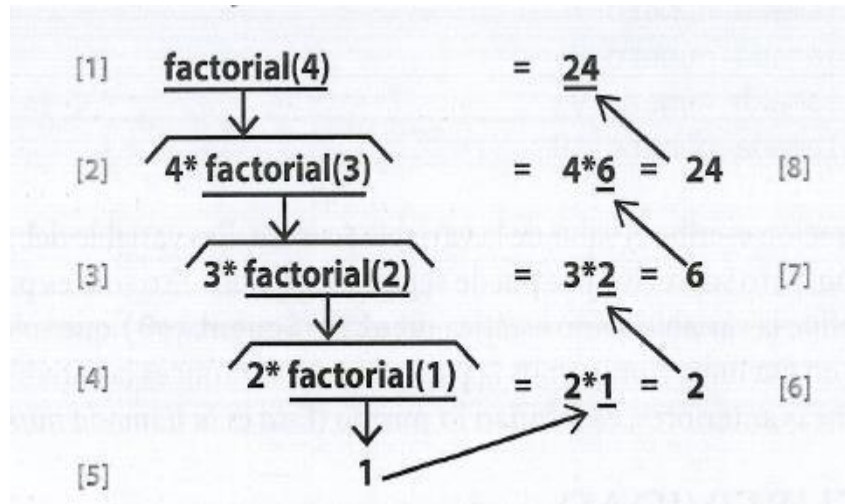
Recursividad

Es una técnica de creación de funciones pensada para solventar problemas complejos. Se basa en que **dentro del código de la función, ésta se invoca a sí misma**

```
function factorial($n){
    if($n<=1) return 1; //condición de salida
    else return $n*factorial($n-1);
}
```

La última instrucción $\$n * \text{factorial}(\$n-1)$ es la que realmente aplica la recursividad. La idea es considerar que el factorial de 9 es 9 multiplicado por el factorial de 8; a su vez el de 8 es 8 por el factorial de 7 y así sucesivamente hasta llegar al factorial de 1, que devuelve 1

EJEMPLO DE FACTORIAL DE 4 CON LA FUNCIÓN ANTERIOR:



- [1] Se llama a la función factorial usando como parámetro el número **4** que será copiado en la variable-parámetro $\$n$
- [2] Como $\$n > 1$, entonces, el resultado será el número **4** multiplicado por el resultado de la llamada **factorial(3)**
- [3] La llamada anterior hace que el nuevo $\$n$ (variable distinta de la anterior) tome el valor **3**, por lo que esta llamada devolverá **3** multiplicado por el resultado de la llamada **factorial(2)**
- [4] La llamada anterior devuelve **2** multiplicado por el resultado de la llamada **factorial(1)**

- [5] Por primera vez el resultado de la llamada recursiva devuelve un resultado directo, el número **1**
- [6] Gracias a ese resultado, la llamada **factorial(2)** da como resultado $2 * 1$, es decir **2**
- [7] Eso hace que la llamada **factorial(3)** devuelva $3 * 2$, es decir **6**
- [8] Por lo que la llamada **factorial(4)** devuelve $4 * 6$, es decir **24** Y ese es ya el resultado final

Cifrado

PHP incorpora numerosas funciones de cifrado. Generalmente, lo que se pretende al cifrar es esconder una contraseña u otros textos que deseamos ocultar a alguien que quiera espiar la comunicación entre nuestro servidor PHP y el cliente. La forma de solucionar el problema es utilizar funciones que convierten la contraseña en un texto **hash** obtenido a través de un algoritmo de cifrado. La idea es que el hash sea indescifrable.

PHP dispone de algoritmos de MD5, SHA1 y SHA256, los cuales no son recomendables. También dispone de Blowfish que sí se recomienda

FUNCIONES CIFRADO (1/2)

```
password_hash(texto, algoritmo [,opciones])
```

- **texto**. Es el texto a cifrar, normalmente una contraseña.
- **algoritmo**. Es un número entero que indica el algoritmo a utilizar. Se utilizan las constantes:
 - **PASSWORD_DEFAULT**. Utiliza el algoritmo **bcrypt** (disponible desde la versión 5.5 de PHP). De modo que utilizará automáticamente el algoritmo de cifrado más robusto que tenga disponible nuestra instalación de PHP.
 - **PASSWORD_BCRYPT**. Utiliza el algoritmo Blowfish. Eso asegura la compatibilidad con la función **crypt** (utilizada tradicionalmente para estos menesteres).
- **opciones**. Se trata de un array asociativo que permite utilizar estas claves:
 - **salt**. Permite indicar una sal manual al obtener el hash. Sin usar esta opción, la función genera una sal aleatoria.
 - **cost**. Coste. Un valor entre 4 y 32 que indica el número de iteraciones (en base 2) que realizará el algoritmo. Por defecto se usa 10. Poner más coste hace que se tarde más en generar la clave (a cambio de más seguridad).

FUNCIONES CIFRADO (2/2)

```
echo password_hash("prueba",PASSWORD_DEFAULT,
    array("salt"=>"123456789ABCDEFGHJKLM"));
```

La función escribe:

\$2y\$10\$123456789ABCDEFGHJKLM.YsIiXtCFs/v7Qo2eLjFKjWUtnfZRpd.

En el propio texto están especificadas las características del algoritmo utilizado:

- **\$2y\$**. Este texto inicial indica el algoritmo utilizado. En este caso es el código del algoritmo **Blowfish**.
- **10\$**. Coste empleado. En este caso 10.
- **123456789ABCDEFGHJKLM**. Sal

■ **YsIiXtCFs/v7Qo2eLjFKjWUtnfZRpd**. Texto hash

FUNCIÓN	RESULTADO
<code>password_get_info(hash)</code>	Recibe un texto hash y nos devuelve un array asociativo con información sobre el texto. En ese array tendremos los siguientes índices: ■ algo. Devuelve un entero relacionado con la constante predefinida que representa el algoritmo utilizado como PASSWORD_DEFAULT por ejemplo ■ algoName. Nombre del algoritmo, por ejemplo bcrypt ■ options. Array de opciones utilizado al cifrar
<code>password_verify(texto,hash)</code>	Devuelve verdadero si el hash es correcto para ese texto. Es la función principal de verificación de una contraseña